Informe Técnico: Componentes Front-End del Módulo de Oferta y Demanda

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Esta evidencia codifica el componente front-end del proyecto **MONK GRAIN** aplicando React, sobre el módulo de **oferta y demanda**: el mercado en su segundo sentido, donde el comprador publica lo que busca y los caficultores responden.

No parte de cero ni inventa una pantalla para la entrega. Parte de algo medido en el repositorio: el módulo tenía **1.359 líneas repartidas en tres páginas y cero componentes reutilizables**, y el proyecto entero repetía el mismo bloque de carga en 28 pantallas. Lo que esta evidencia hace es aplicar el criterio de AA4-EV02 a ese caso.

2. Objetivo

Codificar el componente front-end del proyecto aplicando React, usando los artefactos previos, incluyendo comentarios, cumpliendo los estándares de codificación y versionando el trabajo con Git.

3. El punto de partida, medido

```
features/trading/api.ts                            172 líneas
app/(public)/productor/posturas/page.tsx           221 líneas
app/(public)/mi-cuenta/solicitudes/page.tsx        508 líneas
app/(public)/mi-cuenta/solicitudes/[id]/page.tsx   458 líneas
                                                  ──────────
                                                 1.359 líneas
                                       componentes propios: 0
```

Y la duplicación que atraviesa todo el proyecto:

Patrón repetido	Páginas
Leen el token de sesión a mano	**22**
Repiten el manejo del 403	**14**
Repiten el trío cargando / error / prohibido	**28**

4. Lo construido

4.1 El hook `useProtectedResource`

Encapsula las veinticinco líneas que se repetían: leer el token, redirigir si no hay, pedir el dato, distinguir el 401 del 403, y apagar el indicador de carga.

```
export function useProtectedResource<T>(
  fetcher: (token: string) => Promise<T>,
  deps: readonly unknown[] = [],
): ProtectedResource<T> {
  const router = useRouter();
  const [data, setData] = useState<T | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const [hasError, setHasError] = useState(false);
  const [isForbidden, setIsForbidden] = useState(false);

  const load = useCallback(async () => {
    const token = session.getAccessToken();
    if (!token) { router.push("/ingresar"); return; }

    setHasError(false);
    setIsForbidden(false);

    try {
      setData(await fetcher(token));
    } catch (requestError) {
      if (requestError instanceof ApiError && requestError.isUnauthorized) {
        // La sesión caducó. No es un error que mostrar: es volver a entrar.
        router.push("/ingresar");
        return;
      }
      if (requestError instanceof ApiError && requestError.status === 403) {
        setIsForbidden(true);
        return;
      }
      setHasError(true);
    } finally {
      // En el `finally` y no al final del `try`: sin esto, un error deja la
      // pantalla cargando para siempre. Es el olvido que más se repetía.
      setIsLoading(false);
    }
  }, [router, ...deps]);

  useEffect(() => { void load(); }, [load]);

  return { data, isLoading, hasError, isForbidden, reload: load, setData };
}
```

Las decisiones, con su motivo

Decisión	Por qué
Devuelve **cuatro estados separados** y no uno combinado	No son intercambiables: prohibido es una explicación —el usuario no hizo nada mal—, error ofrece reintentar, y cargando no sabe nada todavía
El **401 no es ninguno de los cuatro**	Significa que la sesión caducó. Lo que corresponde es volver a la pantalla de ingreso, no mostrar un mensaje que el usuario no puede resolver
`setIsLoading(false)` en el `finally`	Fuera de él, un error deja el indicador girando para siempre. Era el olvido más repetido
`deps` explícito y `fetcher` fuera de ellas	`fetcher` se recrea en cada render: depender de él provocaría una petición infinita
Devuelve también `setData`	Permite reflejar un cambio sin volver a pedir todo al servidor

4.2 Cuatro componentes de dominio

Componente	Criterio que cumple	Regla que encierra
`CoverageBar`	Se repite · regla	Una solicitud grande se cubre con varios caficultores. Muestra kilos y porcentaje
`YieldFactorReading`	Regla	Traduce el factor a su lectura comercial. **Nunca muestra el número solo**
`OfferResponse`	Regla	El motivo del rechazo se muestra completo, y la falta de motivo se dice con palabras
`DemandCard`	Se repite	La misma tarjeta para comprador y vendedor, con `href` distinto

Por qué cada uno, en una frase

`CoverageBar`. El porcentaje solo engaña: «80 %» de una solicitud pequeña puede ser menos café que el 20 % de una grande. Y los kilos solos obligan a dividir mentalmente. Los muestra juntos, con la barra marcada como `progressbar` porque un lector de pantalla no ve el ancho de un `div`.

`YieldFactorReading`. Un «88,50» no le dice nada a quien no sabe que el estándar de la Federación es 94 y que menos es mejor. La traducción vive en una sola función compartida con el catálogo: dos lecturas del mismo número en pantallas distintas sería peor que ninguna.

`OfferResponse`. Lo que hace y no haría una etiqueta suelta: muestra el motivo del rechazo completo. Es lo que le dice al caficultor si el problema fue el precio, el factor o la cantidad, y esconderlo tras un «ver más» convierte el rechazo en un «no» sin salida. Cuando no hubo motivo lo dice con palabras, porque un recuadro vacío parece un fallo de la página.

`DemandCard`. El orden de la información sigue la decisión de compra, no el modelo de datos: primero qué busca y cuánto falta, después las condiciones, y el estado al final. Un vendedor decide si postura mirando lo primero.

4.3 La página refactorizada

`/productor/posturas` pasa de 221 a 192 líneas, y lo que desaparece no es maquetación sino el bloque de carga repetido:

```
ANTES                                  DESPUÉS

const [offers, setOffers]   = ...      const { data: page, isLoading,
const [isLoading, ...]      = ...                hasError, isForbidden,
const [error, setError]     = ...                reload }
const [forbidden, ...]      = ...        = useProtectedResource(
                                              (t) => sellerTradingApi.myOffers(t));
const load = useCallback(async () => {
  const token = session.getAccessToken();
  if (!token) { router.push("/ingresar"); return; }
  setError(false);
  try { ... }
  catch (e) { ...401... ...403... }
  finally { setIsLoading(false); }
}, [router]);

useEffect(() => { void load(); }, [load]);
```

5. Estándares aplicados

Estándar (AA1-EV02)	Cómo se cumple
Componentes en `PascalCase`	`CoverageBar.tsx`, `DemandCard.tsx`
Hooks en `camelCase` con prefijo `use`	`useProtectedResource.ts`
Código en inglés, comentarios en español	`remainingKg`, con su porqué en español encima
Tipos explícitos, sin `any`	`tsc` en modo estricto, sin hallazgos
El comentario explica el porqué	«En el *`finally`* y no al final del *`try`*...»
Espejo con el backend	`features/trading/` ↔ `modules/trading/`

6. Pruebas

12 pruebas nuevas, que llevan el frontend de 41 a **53**. Consultan por lo que el usuario percibe —texto accesible, atributos de accesibilidad— y no por la estructura interna que React produjo: un cambio de maquetación no debe romper una prueba que verifica una regla del negocio.

Componente	Qué fijan
`CoverageBar` (5)	Que muestre kilos y porcentaje; que diga cuánto falta mientras no esté cubierta y deje de pedirlo cuando lo esté; que sea legible por voz; y que **no divida por cero**
`YieldFactorReading` (3)	Que **nunca muestre el número solo**; que diga «o mejor» cuando es un techo; y que diga «cualquier factor» cuando no hay requisito
`OfferResponse` (4)	Que el motivo salga completo; que la falta de motivo se diga con palabras; que no hable de motivos si fue aceptada; y que distinga esperar de haber sido rechazada

```
$ npm test

 ✓ tests/navigation.test.ts         (3 tests)
 ✓ tests/quality.test.ts            (11 tests)
 ✓ tests/format.test.ts             (16 tests)
 ✓ tests/TradingComponents.test.tsx (12 tests)   ← nuevas
 ✓ tests/AddToCart.test.tsx         (11 tests)

 Test Files  5 passed (5)
      Tests  53 passed (53)
```

6.1 Las puertas de calidad

```
$ npx tsc --noEmit        sin hallazgos
$ npm run lint            sin hallazgos
$ npm run format:check    All matched files use Prettier code style!
$ npm test                53 passed
$ curl /productor/posturas → 200
```

7. Qué cambió en el proyecto

	Antes	Después
Componentes del módulo `trading`	0	**4**
Hooks propios en el proyecto	0	**1**
Componentes en total	20	**24**
Pruebas de frontend	41	**53**
`/productor/posturas`	221 líneas	192 líneas

> **Las otras 27 páginas siguen repitiendo el bloque.** El hook existe y está probado en una; migrar el resto es trabajo mecánico que conviene hacer por tandas y verificando cada pantalla, no de una sola vez. Queda anotado como pendiente en el README del proyecto, no dado por hecho.

8. Conclusión

El módulo de oferta y demanda pasa de no tener componentes propios a tener cuatro, cada uno justificado por el criterio de AA4-EV02 —se repite, encierra una regla, o tiene estado propio— y ninguno extraído por ser grande.

El hook `useProtectedResource` es lo que mejor ilustra para qué sirven los hooks, que era la pregunta del taller: no dibuja nada, encapsula **comportamiento**, y resuelve una duplicación de veinticinco líneas en veintiocho pantallas que con clases no se habría podido extraer.